
HELMHOLTZ AI
AGENTIC CODING
Or rather everything BUT the coding

WHO ARE WE?

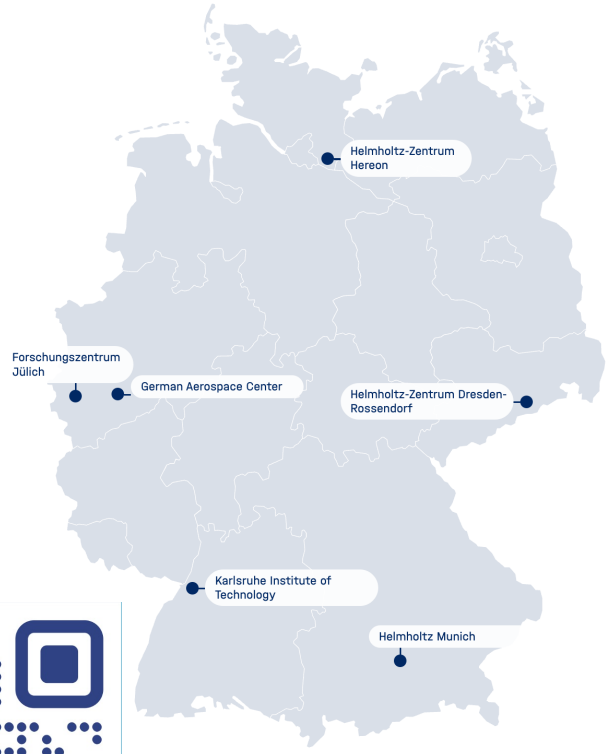
What is our mission?

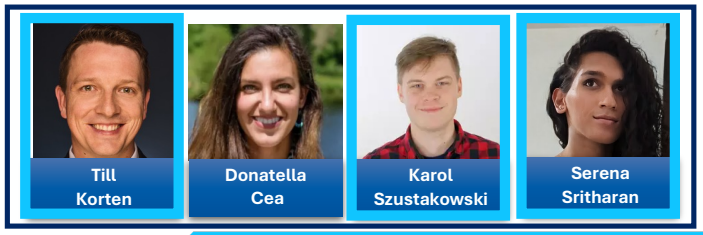
Maximise research impact by democratizing access to AI

How do we do it?

- Short- to mid-term scientific collaboration
- Free of charge
- Easy application

https://voucher-system.helmholtz.ai/





AGENDA

- Intro to agentic coding
- Software engineering practices with Agentic coding
- Setup (OpenCode + Blablador API key)
- Hands-on implementation (+ presentations at the end)

SOFTWARE ENGINEERING DONE RIGHT

Phase	Key activities	Deliverables
1. Planning	Identify project scope, goals and requirements	Initial project plan
2. Analysis	Gather and review data on project requirements	Fully detailed requirement documentation
3. Design	Define project architecture	Software design document (SDD)
4. Coding	Write initial code	Functional software prototype
5. Testing	Review code and eliminate bugs	Refined, optimized software
6. Deployment	Deploy code to production environment	Software available to end users
7. Maintenance	Continual fixes and improvements	Updated and optimized code

Source: www.ibm.com/think/topics/sdlc/jcr%3Acontent

SOFTWARE ENGINEERING DONE RIGHT

Phase	Key activities	Deliverables	
1. Planning	Identify project scope, goals and requirements	Initial project plan	Human input required
2. Analysis	Gather and review data on project requirements	Fully detailed requirement documentation	
3. Design	Define project architecture	Software design document (SDD)	
4. Coding	Write initial code	Functional software prototype	Agentic coding figures this out*
5. Testing	Review code and eliminate bugs	Refined, optimized software	Human input required
6. Deployment	Deploy code to production environment	Software available to end users	
7. Maintenance	Continual fixes and improvements	Updated and optimized code	

Source: www.ibm.com/think/topics/sdlc/jcr%3Acontent

*for the most part

PLANNING

Create your SPEC.md now!

Tip: do not use an LLM to generate it, use it to help you ideate. A brief spec is better than a huge generated one. Make it a source of hard truths!

Check out
https://notes.desy.de/

Problem / project description : What problem are we solving?
Goals & outcomes : What do we want to achieve?
Stakeholders & users : Who is involved or affected?
User needs / use cases : What do users need to do?
Scope : What is included and excluded?
Requirements & constraints : What must we consider from the start?
Feasibility : Can we realistically build it?
Technical approach : What is the initial technical direction?
Risks & assumptions : What could go wrong, and what are we assuming?
Resources & responsibilities : Who does what?
Milestones & timeline : What are the key dates and deliverables?
Development approach : How will we organize the work?
Quality, security & compliance : What standards must we meet?
Success criteria : How will we know we succeeded?

PLANNING

Create your SPEC.md now!

Tip: do not use an LLM to generate it, use it to help you ideate. A brief spec is better than a huge generated one. Make it a source of hard truths!

Problem / project description : What problem are we solving?
Goals & outcomes : What do we want to achieve?
Stakeholders & users : Who is involved or affected?
User needs / use cases : What do users need to do?
Scope : What is included and excluded?
Requirements & constraints : What must we consider from the start?
~~**Feasibility** : Can we realistically build it?~~
Technical approach : What is the initial technical direction?
~~**Risks & assumptions** : What could go wrong, and what are we assuming?~~
~~**Resources & responsibilities** : Who does what?~~
~~**Milestones & timeline** : What are the key dates and deliverables?~~
Development approach : How will we organize the work?
Quality, security & compliance : What standards must we meet?
Success criteria : How will we know we succeeded?

PLANNING

Create your SPEC.md now!

Tip: do not use an LLM to generate it, use it to help you ideate. A brief spec is better than a huge generated one. Make it a source of hard truths!

Problem / project description : What problem are we solving?
Goals & outcomes : What do we want to achieve?
Stakeholders & users : Who is involved or affected?
User needs / use cases : What do users need to do?
Scope : What is included and excluded?
Requirements & constraints : What must we consider from the start?
Technical approach : What is the initial technical direction?
Development approach : How will we organize the work?
Quality, security & compliance : What standards must we meet?
Success criteria : How will we know we succeeded?

ANALYSIS

We will shorten the analysis stage today, let's focus just on the user stories.

Let's write them one person at a time. Keep them detailed:

Good example: I click the "render" button at the bottom of the interface, and the image is rendered on top of the button in color with a brightness slider to the right".

Bad example: I click render and can see the image and adjust the brightness.

If you want to include more details here:

What performance, reliability, security, etc. are required? → known as non-functional requirements
What data is needed, produced, and stored?
What other systems or components are involved?
What technical, organizational, or regulatory limits apply?
What functions and behaviors are needed? → known as functional requirements

DESIGN – TECH STACK

Drop your SPEC.md into an agent, and ask it:

- Which coding frameworks can be used to achieve this (frontend, backend)?
- What are the considerations for each of the frameworks. What are the limitations?
- What is the best pipeline for testing and creating this software with you if my coding expertise is [your coding expertise]?
- Which framework would you recommend, and why?

Do not let the agent implement JavaScript or Python or [your language of choice] from scratch! Use a framework!

DESIGN

Give it a read and decide on:

- your frontend and backend frameworks (if they apply),
- your UI/UX frameworks (if they apply),
- your testing setup,
- your build and deployment setup.

DESIGN (PART OF SPEC.MD)

Technical Stack
Frontend
Next.js
Backend
FastAPI
UI/UX
Tailwind CSS + component library – material tailwind
Testing
Pytest– no end-to-end tests here
Build
Local – no docker or other containers for this prototype
Deployment
Local

DESIGN (PART OF SPEC.MD)

Development framework
Use the waterfall model for this software and implement everything end-to-end.

OR

Development framework
Use an agile for this software and include me in feedback loops often. Implement one feature at a time.

OR

Development framework
Use the RAD model, deliver the first working prototype as soon as possible.

HOW TO SPLIT YOUR WORK IF THE AGENT DOES THE CODING?

Team Collaboration

- Create a GitHub repository and upload your SPEC.md (and other relevant files) there.
- Keep a TODO.md in the repository.
- Continuously test the app while the agent is developing it.
- Add bugs, requests, and observations to TODO.md.
- For larger teams, use one TODO file per person, e.g. TODO_Matthias.md, TODO_Mike.md → No git? No problem! Just send your files to the person running the agent.

Tell the agent (in SPEC.md):

“Team members will add requests and bugs to the TODO files in this repository. Check them after each major change. When you find new items, address them where appropriate. Do not modify the TODO files, keep your own DONE.md to mark finished tasks.”

~~TIME TO PRESS PLAY!~~

Take your finished SPEC.md to your agent, ask it to read it and ask it **to ask you any questions which would clarify anything about the project which is yet unclear.**

OUR TIP

Whenever possible, use Test-Driven-Development (TDD). Ask the agent to write tests first, and only then proceed with writing the software. Without end-to-end tests, this might be difficult for web apps, but the core functions can still usually be tested with unit tests.

An idea for this workshop:
“Start with a TDD setup, write tests which do not require a long time to run but will verify the core logic. Run the tests and verify that they fail. Do not use end-to-end tests which require long runtimes. Aim for a high test coverage”.

TIME TO PRESS PLAY!

Take your finished SPEC.md to your agent, ask it to read it and ask it **to ask you any questions which would clarify anything about the project which is yet unclear.**

After the questions are answered, ask the agent to start working. You can put some relevant answers in a FAQ section of your SPEC.md, too.

You just learned spec-driven-development!

QUESTIONNAIRE

- Please take a couple of minutes to provide feedback



HELMHOLTZAI | ARTIFICIAL INTELLIGENCE COOPERATION UNIT



THANK YOU

RESOURCES

- Introductory slides : https://github.com/Helmholtz-AI-Matter/coding-assistants
- Slides on hands-on session : https://github.com/HelmholtzAI-Consultants-Munich/agentic-coding-hands-on
- Workshop page (shared with participants) : https://notes.desy.de/OZCosgehTVeEJWbqQ3quwg?view